Dokumentation der Praktischen Arbeit zur Prüfung zum Mathematisch-technischen Softwareentwickler

08.05.2026

Mohammed Alassane

Prüfungsnummer: 142 38001

Programmiersprache: Java

Inhaltsverzeichnis

1. Aufgabenanalyse 3
1.1 Interpretation der Aufgabe 3
1.2.1 Einlesen der Daten 4
1.2.2 Kollisionsprüfung 5
1.2.3 Lösungsstrategien 6
1.2.4 Ausgabe der Ergebnisse 7
1.2.5 Fehlerbehandlung 8
1.2.6 Erläuterung anhand von Beispielen 8
1.2.7 Nicht-funktionale Anforderungen 12
2. Entwicklerdokumentation 2.1 Projektstruktur 13
2.2 Javadocs 14
2.3 UML-Klassendiagramm 14
2.4 UML-Sequenzdiagramm 15
2.5 Aktivitätsdiagramme 16
2.5.1 Einseitiges Warten 17
2.5.2 Beidseitiges Warten 17
2.5.3 Beidseitiges Warten 18
3. Testdokumentation 20
3.1 Diskussion der IHK-Beispiele 20
3.2 Eigene Beispiele 20
3.3 Grenz- und Fehlerfälle 21
3.3.1 Fehlerfälle 21
3.3.2 Grenzfälle 22
4. Abschließende Bemerkungen 4.1 Zusammenfassung 23
4.2 Ausblick 23
4.3 Abweichungen und Ergänzungen zum Vorentwurf 24
4.3.1 Ergänzungen 24
4.3.2 Abweichungen 24
4.4 Benutzeranleitung 25
4.5 Verwendete Hilfsmittel 26
4.6 Entwicklungsumgebung 26
4.7 Eigenhändigkeitserklärung 27

1. Aufgabenanalyse

1.1 Interpretation der Aufgabe

Die MATSE-Eisenbahngesellschaft betreibt eingleisige Bahnlinien, die sich aus Teilstrecken, die durch Bahnhöfe verbunden sind, zusammensetzt. Dadurch, dass die Strecken eingleisig sind, dürfen sich Züge nur an Bahnhöfen begegnen, da es sonst zu Kollisionen kommt. Die Züge fahren dabei alle 60 Minuten von einem Bahnhof ab wodurch sich ein periodischer Ablauf ergibt

Die zentrale Aufgabe ist es ein Softwaresystem zu entwickeln, dass auf Grundlage verschiedener Strategien Fahrtpläne für diese Eisenbahnlinien erstellt.

Die Züge sollen bei Ankunft an einem Bahnhof eine anpassbare Einstiegszeit verweilen, um Passagieren den Umstieg zu ermöglichen und vor der Abfahrt einen Sicherheitsabstand zur vorhergien entgegenkommenden Fahrt einhalten.

Die Fahrpläne sollen anhand dreier verschiedener Strategien ermittelt werden:

1. Bei der einfachen Fahrt sollen die Züge so schnell wie möglich von Bahnhof zu Bahnhof fahren, ohne dabei auf Kollisionen zu achten.

2. Bei der Strategie des einseitigen Wartens werden Fahrpläne ermittelt, bei denen Kollisionen verhindert werden, indem die Züge auf der Rückfahrt ggfs. warten müssen, bis der Hinzug sicher in den Bahnhof eingefahren ist. Dadurch werden alle Wartezeiten auf die Rückfahrt konzentriert.

3. Bei der Strategie des beidseitigen Wartens soll die Wartezeit auf Hin- und Rückfahrt aufgeteilt werden, um für die Kunden ein angenehmeres Fahrterlebnis zu schaffen.

Die Güte der Fahrpläne soll durch den sog. „Score“ gemessen werden, der sich aus der Summe der quadrierten Gesamtwartezeiten für Hin- und Rückweg ergibt.

Die ermittelten Fahrpläne sollen am Ende zusammen mit ihren Kennzahlen ausgegeben werden.

1.2.1 Einlesen der Daten

Das Einlesen der Daten funktioniert über standardisierte .txt Eingabedateien mit dem folgenden Format:

// Beispiel 1

Strecke:
A B C

Abstaende:
5 7

Start Hinfahrt:
17

Die Eingabedateien bestehen aus einem optionalen Kommentar, gefolgt von drei Abschnitten, in denen die Bahnhofsnamen, Fahrtdauern zwischen den Bahnhöfen und der Startzeitpunkt der Hinfahrt angegeben werden.

Die Bahnhofsnamen werden als Strings beliebiger Länge und durch Leerzeichen separiert angegeben. Die Abstände zwischen den Bahnhöfen werden der Reihenfolge der Bahnhöfe entsprechend als ganzzahlige Werte in Minuten, ebenfalls durch Leerzeichen getrennt, definiert. Die Startzeit der Hinfahrt wird als einzelner ganzzahliger Wert übergeben.

Die Eingabeverarbeitung (ConcreteInputHandler.java) öffnet die Text-Eingabedatei und bereinigt diese zunächst, wobei überflüssige Leerzeilen und Kommentare, die durch „//“ gekennzeichnet sind, herausgefiltert werden.

Nach der Bereinigung extrahiert der Parser die Nutzdaten aus den exakt vorgegebenen Zeilen. Dabei findet eine strikte Datenvalidierung statt, um die Konsistenz des Fahrplans zu gewährleisten:

1. **Plausibilität der Streckenabschnitte:** Es wird geprüft, ob mindestens zwei Bahnhöfe existieren und ob die Anzahl der angegebenen Streckenabstände exakt um eins kleiner ist als die Anzahl der Bahnhöfe.
2. **Datentyp-Prüfung:** Die Fahrtdauern und die Startzeit müssen als gültige ganzzahlige Werte vorliegen.
3. **Plausibilität der Fahrtzeiten:** Die Fahrtdauern müssen mindestens eine Minute betragen.
4. **Physische Machbarkeit:** Da ein 60-Minuten-Takt vorliegt und Züge nur an Bahnhöfen kreuzen dürfen, darf ein einzelner Streckenabschnitt inklusive der vorgeschriebenen Sicherheitswartezeit eine Dauer von 29 Minuten nicht überschreiten, da Züge ansonsten auch mit beliebiger Wartezeit kollidieren würden.
5. **Startzeit der Hinfahrt:** Die Startzeit der Hinfahrt muss im Intervall [0; 59] liegen.

Sind alle Prüfungen erfolgreich, instanziiert der Input-Handler aus den gelesenen Daten die entsprechende Anzahl an Bahnhof- und Strecke-Objekten, verknüpft diese in der logischen Reihenfolge der Hinfahrt, setzt die Startzeit am Initialbahnhof und gibt die fertige Streckenliste zurück.

Sollte es beim Einlesen der Datei, beim Parsen der Werte oder während der Konsistenzprüfungen zu Fehlern kommen, wird eine spezifische InputHandlerException mit genauer Fehlerbeschreibung geworfen, die zentral behandelt wird.

1.2.2 Kollisionsprüfung

Zur Kollisionsprüfung einer Teilstrecke wird in der Klasse „Strecke“ die Methode „public boolean pruefeKollision()“ verwendet. Sie untersucht ob auf der jeweiligen Strecke eine Kollision vorliegt.

Da in dem zu modellierenden System Züge in einem 60-Minuten-Takt an Bahnhöfen einfahren, genügt ein simpler Abgleich von absoluten Start- und Endzeiten nicht. Stattdessen wendet der Algorithmus zunächst Modulo-Arithmetik (% 60) an, um die Abfahrtszeiten beider Züge auf den Rahmen einer relativen Stunde zu normieren. Um zu verhindern, dass Zeitintervalle, die über einen Stundenwechsel hinausgehen (beispielsweise eine Abfahrt in Minute 58 und eine Ankunft 4 Minuten später), mathematisch in zwei separate Teile zerrissen werden, wird die Fahrtdauer erst nach der Modulo-Operation auf die normierte Startzeit addiert. So entstehen logisch kontinuierliche Intervalle (z. B. [58, 62]), mit denen das System fehlerfrei rechnen kann.

Des Weiteren muss die periodische Eigenschaft des Fahrplans bei der Kollisionsprüfung berücksichtigt werden. Durch den kontinuierlichen Stundentakt kann ein Zug der aktuellen Stunde theoretisch mit drei verschiedenen Gegenzügen in Konflikt geraten: mit der Rückfahrt der exakt gleichen Stunde, der vorherigen Stunde oder der nächsten Stunde. Um all diese Fälle abzudecken, verschiebt die Logik das Intervall des Gegenzugs rechnerisch um jeweils 60 Minuten nach vorn sowie nach hinten und prüft alle drei potenziellen Kollisionsszenarien systematisch ab.

Die eigentliche Überschneidungsprüfung erfolgt durch einen Abgleich der Start- und Endpunkte der Zugintervalle. Hierbei wird auch die Einhaltung des Sicherheitsabstands kontrolliert. Die Bedingung prüft mithilfe von Inklusiv-Operatoren (<=), ob der Startzeitpunkt des einen Zuges kleiner oder gleich der Ankunftszeit des anderen Zuges ist. Dadurch wird erkannt, ob ein Zug aus einem Bahnhof abfahren will, während die Sicherheitszeit noch nicht abgelaufen ist. Sobald in auch nur einem der drei betrachteten 60-Minuten-Takte eine Überschneidung festgestellt wird, setzt die Methode das Kollisions-Flag der Strecke auf „true“ und gibt „true“ als Wiedergabewert zurück.

1.2.3 Lösungsstrategien

1.2.3.1 Einfache Fahrt

Die Lösungsstrategie „Einfache Fahrt“ repräsentiert das theoretische Best-Case-Szenario der Fahrzeitplanung. Das Kernmerkmal dieses Algorithmus ist das strikte „ASAP“-Prinzip (As Soon As Possible): Züge fahren schnellstmöglich von Bahnhof zu Bahnhof und halten sich lediglich exakt an die vorgeschriebene Einstiegszeit. Künstliche Wartezeiten zur Konfliktauflösung existieren hier nicht und auch die Sicherheitswartezeit wird nicht eingehalten.

Zur Vorbereitung erstellt die Methode zunächst eine tiefe Kopie der Streckenliste, um Seiteneffekte auf die Ursprungsdaten zu verhindern. Zudem werden die Randbahnhöfe (der allererste und der allerletzte Bahnhof der Gesamtstrecke) logisch initialisiert, da der erste Bahnhof keine vorherige Ankunftszeit und der letzte keine abzweigende Abfahrtszeit besitzt. Dasselbe gilt für die Rückfahrt, wobei da der erste Bahnhof und der letzte Bahnhof die Rillen tauschen. Die Fahrplanermittlung gliedert sich anschließend in zwei Phasen:

In der ersten Phase wird die Hinfahrt vom Start- bis zum Endbahnhof berechnet. Für jeden Streckenabschnitt kalkuliert der Algorithmus die Ankunftszeit am Zielbahnhof (Abfahrtszeit des Startbahnhofs plus Fahrtdauer). Die anschließende Abfahrtszeit ergibt sich das hinzufügen der Einstiegszeit. Um die Stationen logisch miteinander zu verknüpfen, werden am Ende jedes Schleifendurchlaufs die berechneten Zeiten des aktuellen Zielbahnhofs direkt in das Objekt des Startbahnhofs der darauffolgenden Strecke kopiert. So wird die Zeitachse lückenlos fortgeschrieben.

In der zweiten Phase werden die Fahrtzeiten der Rückfahrt in umgekehrter Richtung, also vom chronologisch Ersten bis zum chronologisch letzten Bahnhof, berechnet. Sie startet frühestmöglich nach der Ankunft des Hinfahrt-Zuges am letzten Bahnhof und nach Ablaufen der Einstiegszeit. Innerhalb der Rückwärts-Schleife wird für jeden Streckenabschnitt die Zeitberechnung durchgeführt. Dabei ist die Ankunftszeit der Rückfahrt die Abfahrt am vorherigen Bahnhof plus Fahrtdauer und die neue Abfahrtszeit ist wiederum die Ankunft plus Einstiegszeit. Auch hier erfolgt am Ende jeden Schrittes die Übergabe der berechneten Zeiten an die nächste rückwärtsgerichtete Iteration.

Am Ende jedes Iterationsschritts der Rückfahrt wird die Strecke auf Kollisionen geprüft. Im Falle einer Kollision wird das Kollisions-Flag der Strecke auf „true“ gesetzt, aber sonst wird die Kollision nicht weiter behandelt.

Abschließend wird der fertige Fahrplan als Liste von Strecke-Objekten zurückgeliefert.

1.2.3.2 Einseitiges Warten

Die Lösungsstrategie des einseitigen Wartens implementiert einen asymmetrischen Ansatz zur Konfliktauflösung bei Kollisionen.

Zunächst teilt sich der Algorithmus die Schritte mit der Strategie der einfachen Fahrt. Die Unterschiede treten dann in Phase 2 auf. Während der Rückwärts-Iteration über die Strecken, werden wie bei der einfachen Fahrt, die Strecken auf Kollisionen geprüft. Falls eine Kollision gefunden wird, wird allerdings jetzt eine Korrektur vorgenommen. Der Rückfahrtszug, muss warten bis der Hinfahrtszug in den Bahnhof eingetroffen ist und die Sicherheitswartezeit abgelaufen ist. Daraus folgt, dass die Abfahrtszeit des Rückzuges angepasst wird. Wie in dem zuvor erklärten Algorithmus werden nun die berechneten Fahrtzeiten in den Iterationsfolger eingetragen. Nachdem die gesamte Bahnlinie berechnet wurde, wird der Fahrplan, der nun ggfs. Wartezeiten auf der Rückfahrt enthält, zurückgegeben.

1.2.3.3 Beidseitiges Warten

Die Strategie des beidseitigen Wartens verwendet zunächst die Strategie der einfachen Fahrt und prüft dann, ob Kollisionen vorliegen. Sollte das nicht der Fall sein, wird der Fahrplan in dieser Form zurückgeliefert. Sollten Kollisionen vorhanden sein, wird versucht den Startzeitpunkt der Rückfahrt zu verzögern. Sollte sich dabei eine Startzeit ergeben, bei der keine Kollisionen mehr vorkommen, wird diese Lösung wiedergegeben.

Ansonsten beginnt der komplexe Lösungsalgorithmus. Der Algorithmus ist ein Greedy-Relaxation-Algorithmus. Zunächst wird nach der Strategie des einseitigen Wartens eine Mindestlösung ermittelt, die es zu schlagen gilt. Diese dient gleichzeitig auch als Fallback, sollte der Greedy-Relaxation-Algorithmus zu keinem stabilen Ergebnis führen. Ein Fahrplan gilt dann als stabil, wenn keine Kollisionen mehr vorkommen.

Der Algorithmus beginnt mit einer Schleife, über alle möglichen Startzeitpunkte der Rückfahrt und berechnet für diesen Startzeitpunkt den Fahrplan mit Kollisionen. Dann beginnt eine Schleife in der chronologisch von der ersten Strecke bis zur letzten Strecke die erste vorkommende Kollision gesucht wird. Wurde die Kollision gefunden wird anhand einer Kostenfunktion nach der günstigsten Korrektur gesucht, d.h. es wird entweder auf der Hinfahrt oder auf der Rückfahrt Wartezeit an dieser Stelle eingebaut. Die Kostenfunktion bewertet diejenige Wahl als günstiger bei der die Wartezeit, die bei der Korrektur einzufügen ist, geringer ist, wobei ein Ungleichgewicht zwischen der Gesamtwartezeiten der Hin- und Rückfahrt mit eingerechnet wird. Dieser Prozess wieder wiederholt, bis ein stabiler Fahrplan ermittelt wurde oder die Maximalzahl an versuchen für die aktuelle Verschiebung erreicht wurde. Sollte ein stabiler Fahrplan gefunden werden, wird dieser mit dem aktuelle Bestergebnis verglichen, welches ggfs. dann überschrieben wird. Am Ende wird der errechnete Fahrplan zurückgegeben.

Da der Greedy-Relaxation-Algorithmus nicht unbedingt eine Lösung findet, wird das Ergebnis des einseitigen Wartens als Notlösung verwendet.

Im Worst-Case muss der Algorithmus die Strategien des einfachen Fahrens und des einseitigen Wartens anwenden, welche jeweils eine Laufzeitkomplexität von O(n) haben. Zusammen mit der Laufzeitkomplexität des Greedy-Relaxation Algorithmus ergibt sich insgesamt eine Worst-Case Laufzeitkomplexität von O(n).

1.2.4 Ausgabe der Ergebnisse

Die Ausgabeverarbeitung übernimmt die Ausgabe der ermittelten Fahrpläne. Zunächst stellt die Logik sicher, dass ein Zielverzeichnis „output/“ existiert und legt dieses bei Bedarf an.

Der Ausgabe-Methode wird der Dateiname der Eingabe übergeben, damit eine Ausgabe-datei unter „output/output_<eingabe>.txt“ erstellt werden kann. In diese Datei wird dann die Ausgabe geschrieben.

Die Ausgabe beginnt mit einem Header, welcher sowohl den Inhalt der Eingabe als auch die Kennzahlen „Anzahl Bahnhöfe“ und „Mindestdauer“ enthält.

Im Hauptteil der Datei werden die Resultate der verschiedenen Lösungsstrategien iterativ abgearbeitet und übersichtlich in Tabellenform dargestellt. Für jede Strategie wird ein eigener Abschnitt generiert. Jeder Abschnitt führt die Ausgabe für eine Liste an Strecken durch, wobei darauf geachtet wird, dass die Bahnhöfe nicht doppelt ausgegeben werden.

Ein Abschnitt ist nach Vorgabe der Aufgabestellung aufgebaut, wobei die Wartezeiten lokal vom OutputHandler berechnet werden. Nach der Tabelle mit den Fahrtzeiten eines Absatzes werden die Kennziffern ausgegeben und die Ausgabe geht mit der nächsten Lösung, falls vorhanden, weiter.

1.2.5 Fehlerbehandlung

Falls während der Verarbeitung Fehler auftreten sollten, werden diese gefangen und in der Konsole gesammelt mit entsprechenden Fehlernachrichten ausgegeben.

1.2.6 Erläuterung anhand von Beispielen

1.2.6.1 Einfache Fahrt

Um die Funktionsweise der einfachen Fahrt sowie der Kollisionsprüfung praxisnah zu veranschaulichen, wird im Folgenden ein stark vereinfachtes Streckennetz betrachtet. Die Route besteht aus den drei Bahnhöfen A, B und C. Die Fahrtdauer zwischen A und B beträgt 25 Minuten, zwischen B und C sind es 19 Minuten. Die Einstiegszeit und der Sicherheitsabstand sind systemweit auf eine Minute festgelegt. Die Hinfahrt startet planmäßig zur Minute 12. Die passende Eingabedatei findet sich unter „input/Beispiel_Erklaerung_1.txt“

Algorithmus der einfachen Fahrt berechnet zunächst einen Fahrplan nach dem ASAP-Prinzip (As Soon As Possible). Die Hinfahrt beginnt in Bahnhof A um Minute 12. Nach 25 Minuten Fahrzeit erreicht der Zug den Bahnhof B zur Minute 37. Es folgt die einminütige Einstiegszeit, sodass die planmäßige Abfahrt in B auf Minute 38 fällt. Nach weiteren 19 Minuten erreicht die Hinfahrt ihren Endbahnhof C zur Minute 57.

Unmittelbar im Anschluss wird die Rückfahrt in entgegengesetzter Richtung kalkuliert. Der Zug startet frühestmöglich nach Ankunft der Hinfahrt zuzüglich der Einstiegszeit, verlässt den Bahnhof C also zur Minute 58. Die Fahrt zu Bahnhof B dauert 19 Minuten, was rechnerisch zu Minute 77 führt. Durch die Normierung auf den Stundentakt (Modulo 60) ergibt sich als Ankunftszeit in B die Minute 17.

Nun wird die erste Kollisionsprüfung für den gerade berechneten Streckenabschnitt zwischen C und B durchgeführt. Die Logik vergleicht das normierte Zeitintervall der Hinfahrt [38, 57] mit dem mathematischen Intervall der Rückfahrt [58, 77]. Die Prüfung kontrolliert, ob die Abfahrt des einen Zuges kleiner oder gleich der Ankunft des Gegenzuges ist. Da die Rückfahrt in C erst startet (Minute 58), nachdem die Hinfahrt bereits sicher angekommen ist (Minute 57), überschneiden sich die Intervalle in keinem der drei betrachteten Stundentakte. Der Abschnitt zwischen B und C wird folglich als kollisionsfrei gewertet.

Nach der Einstiegszeit erfolgt die Abfahrt in B zur Minute 18, woraufhin der Zug nach 25 Minuten Fahrzeit den Startbahnhof A zur Minute 43 erreicht.

Nun wird auch diese letzte Strecke (A nach B) auf Kollisionen geprüft. Das Intervall der Hinfahrt erstreckt sich hier von der Abfahrt in A (Minute 12) bis zur Ankunft in B (Minute 37) – mathematisch repräsentiert als [12, 37]. Die Rückfahrt plant, denselben Streckenabschnitt in der Gegenrichtung von der Abfahrt in B (Minute 18) bis zur Ankunft in A (Minute 43) zu belegen, was das Intervall [18, 43] ergibt. Der Algorithmus gleicht diese Intervalle nun ab: Da die Abfahrt der Hinfahrt (12) kleiner als die Ankunft der Rückfahrt (43) ist und gleichzeitig die Abfahrt der Rückfahrt (18) deutlich vor der Ankunft der Hinfahrt (37) stattfinden soll, schlägt die Inklusiv-Bedingung an. Anschaulich bedeutet das, dass sich der Hinzug noch auf der offenen Strecke befindet, während der Gegenzug bereits in Bahnhof B losfährt. Die Kollision wird erfasst und die Strecke A-B wird mit einem Kollisions-Flag markiert.

1.2.6.2 Einseitiges Warten

Zur Demonstration der Strategie des einseitigen Wartens wird ebenfalls das Beispiel „input/Beispiel_Erklaerung_1.txt“ verwendet.

Zunächst wird die Hinfahrt über die Bahnlinie nach dem ASAP-Prinzip berechnet. Die Hinfahrt beginnt in Bahnhof A um Minute 12. Nach 25 Minuten Fahrzeit erreicht der Zug den Bahnhof B zur Minute 37. Es folgt die einminütige Einstiegszeit, sodass die planmäßige Abfahrt in B auf Minute 38 fällt. Nach weiteren 19 Minuten erreicht die Hinfahrt ihren Endbahnhof C zur Minute 57.

Unmittelbar im Anschluss wird die Rückfahrt in entgegengesetzter Richtung kalkuliert. Der Zug startet frühestmöglich nach Ankunft der Hinfahrt zuzüglich der Einstiegszeit, verlässt den Bahnhof C also zur Minute 58. Die Fahrt zu Bahnhof B dauert 19 Minuten, was rechnerisch zu Minute 77 führt. Durch die Normierung auf den Stundentakt (Modulo 60) ergibt sich als Ankunftszeit in B die Minute 17.

Nun wird die erste Kollisionsprüfung für den gerade berechneten Streckenabschnitt zwischen C und B durchgeführt. Die Logik vergleicht das normierte Zeitintervall der Hinfahrt [38, 57] mit dem mathematischen Intervall der Rückfahrt [58, 77]. Die Prüfung kontrolliert, ob die Abfahrt des einen Zuges kleiner oder gleich der Ankunft des Gegenzuges ist. Da die Rückfahrt in C erst startet (Minute 58), nachdem die Hinfahrt bereits sicher angekommen ist (Minute 57), überschneiden sich die Intervalle in keinem der drei betrachteten Stundentakte. Der Abschnitt zwischen B und C wird folglich als kollisionsfrei gewertet.

Nach der Einstiegszeit erfolgt die Abfahrt in B zur Minute 18, woraufhin der Zug nach 25 Minuten Fahrzeit den Startbahnhof A zur Minute 43 erreicht.

Nun wird auch die Strecke zwischen A und B auf Kollisionen untersucht. Das Intervall der Hinfahrt erstreckt sich hier von Minute 12 (Abfahrt A) bis Minute 37 (Ankunft B). Die neu berechnete Rückfahrt würde denselben Streckenabschnitt von Minute 18 (Abfahrt B) bis Minute 43 (Ankunft A) belegen. Da sich diese Zeitfenster überschneide, wird eine Kollision festgestellt.

An dieser Stelle wird nun die Kollision korrigiert, indem auf der Rückfahrt an Bahnhof B Wartezeit eingebaut wird. Der Algorithmus ermittelt, wann der Streckenabschnitt wieder sicher befahrbar ist. Da die Hinfahrt zur Minute 37 im Bahnhof B eintrifft und der zwingende Sicherheitsabstand von einer Minute greift, ist das Gleis für die Gegenrichtung erst zur Minute 38 wieder befahrbar.

Die geplante Abfahrt der Rückfahrt am Bahnhof B wird nun v on Minute 18 auf Minute 38, nach hinten, verschoben. Dadurch entsteht eine Wartezeit von exakt 20 Minuten.

1.2.6.3 Beidseitiges Warten

Die Strategie des beidseitigen Wartens wendet ein mehrstufiges Verfahren an. Zur Demonstration der zweiten Lösungsstufe, also der Lösung durch Verschiebung der Rückfahrt, wird das Beispiel „input/Beispiel_Erklaerung_2.txt“ verwendet. Die Route besteht aus den drei Bahnhöfen A, B und C. Die Fahrtdauer zwischen A und B sowie zwischen B und C beträgt jeweils 20 Minuten. Die Einstiegszeit und der Sicherheitsabstand sind auf eine Minute festgelegt. Die Hinfahrt startet planmäßig zur Minute 0.

Der Algorithmus berechnet zunächst einen Fahrplan nach dem ASAP-Prinzip der einfachen Fahrt. Die Hinfahrt beginnt in Bahnhof A um 0. Nach 20 Minuten Fahrzeit erreicht der Zug den Bahnhof B zur Minute 20. Es folgt die einminütige Einstiegszeit, sodass die planmäßige Abfahrt

in B auf Minute 21 fällt. Nach weiteren 20 Minuten erreicht die Hinfahrt ihren Endbahnhof C zur Minute 41. Im Anschluss beginnt die Rückfahrt in Richtung Bahnhof A. Der Zug startet frühestmöglich nach Ankunft der Hinfahrt zuzüglich der Einstiegszeit, verlässt den Bahnhof C also zur Minute 42. Die Fahrtdauer zum Bahnhof B beträgt 20 Minuten, wodurch sich als Ankunftszeit in B die Minute 2 ergibt. Nach der Einstiegszeit erfolgt die Abfahrt in B zur Minute 3.

Der fertig berechnete Streckenabschnitt zwischen B und C wird nun auf Kollision überprüft. Die Logik vergleicht das normierte Zeitintervall der Hinfahrt [21, 41] mit dem mathematischen Intervall der Rückfahrt [42, 62]. Da wieder die Abfahrt noch die Ankunft des Zuges im Intervall des Gegenzuges liegt, findet keine Kollision statt und es wird der nächste Abschnitt untersucht.

Der Streckenabschnitt zwischen A und B für die Rückfahrt wird berechnet und auf Kollisionen geprüft. 20 Minuten nach Abfahrt erreicht der Rückzug den Startbahnhof A zur Minute 23. Nun wird auch diese letzte Strecke auf Kollisionen geprüft. Das Intervall der Hinfahrt erstreckt sich hier von der Abfahrt in A zur Minute 0 bis zur Ankunft in B in der Minute 20. Die Rückfahrt plant, denselben Streckenabschnitt in der Gegenrichtung von der Abfahrt in B um 3 nach bis zur Ankunft in A zur Minute 23 zu belegen, was das Intervall [3, 23] ergibt. Der Algorithmus gleicht diese Intervalle nun ab: Da die Abfahrt der Hinfahrt (0) kleiner als die Ankunft der Rückfahrt (23) ist und gleichzeitig die Abfahrt der Rückfahrt (3) deutlich vor der Ankunft der Hinfahrt (20) stattfinden soll, schlägt die Inklusiv-Bedingung an. Anschaulich bedeutet das, dass sich der Hinzug noch auf der offenen Strecke befindet, während der Gegenzug bereits in Bahnhof B losfährt. Die Kollision wird erfasst und die Strecke A-B wird mit einem Kollisions-Flag markiert.

Nun beginnt die zweite Lösungsstufe, in der versucht wird die Kollisionen durch Verschiebungen aufzulösen. Es wird von 0-59 über jede mögliche Verschiebung des Starts der Rückfahrt iteriert. Dabei wird für jede Version ein neuer Fahrplan nach dem obigen Schema berechnet. Mit den geänderten Rückfahrtzeiten wird in jeder Iteration untersucht ob Kollisionen vorliegen. Bei der Kollisionsprüfung mit einer Verschiebung von 18 Minuten ergeben sich die folgenden Zeitintervalle: Auf der Strecke B – C hat die Hinfahrt das Intervall [21; 41] und die Rückfahrt das Intervall [0; 20]. Anschaulich treffen die Züge hier sicher im Bahnhof aufeinander, es entsteht also keine Kollision. Die Strecke A – B hat auf der Hinfahrt das Intervall [0; 20] und auf der Rückfahrt das Intervall [21; 41]. Eine Kollisionsprüfung ergibt, dass der Fahrplan stabil ist. Da keine Kollisionen und keine Wartezeiten vorliegen erreicht diese Lösung einen perfekten Score von 0.

Um den Greedy-Relaxation-Algorithmus der Strategie zu demonstrieren, liegt unter „input/Beispiel_Erklaerung_3.txt“ die folgende Problemsituation vor:

Die Route besteht aus den vier Bahnhöfen A, B, C und D. Die Fahrtdauer zwischen A und B sowie zwischen B und C beträgt jeweils 20 Minuten, zwischen C und D sind es 15 Minuten. Die Einstiegszeit und der Sicherheitsabstand sind auf eine Minute festgelegt. Die Summe aus Fahrtdauer und Sicherheitswartezeit überschreitet auf keiner Strecke die Grenze von 30 Minuten. Die Hinfahrt startet planmäßig zur Minute 0.

Der Algorithmus berechnet zunächst einen Fahrplan nach dem ASAP-Prinzip. Da das Streckennetz stark ausgelastet ist, entsteht eine Kollision im Streckenabschnitt B – C. In der zweiten Stufe der Lösungsstrategie werden alle Verschiebungen der Rückfahrt ausprobiert, um die Kollisionen zu beheben. Allerdings führt in diesem Szenario keine reine Verschiebung zu einem kollisionsfreien Ergebnis. Der Algorithmus geht somit in die dritte Lösungsstufe über.

Zunächst wird nach der Strategie des einseitigen Wartens eine Mindestlösung ermittelt. In diesem Fall entsteht eine Wartezeit von 28 Minuten im Bahnhof C sowie 18 Minuten im Bahnhof B auf der Rückfahrt. Dies führt zu einem Score von 2116 (46^2 = 2116). Diese Lösung dient gleichzeitig auch als Fallback, sollte der Greedy-Relaxation-Algorithmus zu keinem stabilen Ergebnis führen.

Nun beginnt der komplexe Lösungsalgorithmus der Greedy-Relaxation. Eine Schleife iteriert über alle möglichen Startzeitpunkte der Rückfahrt und berechnet für diesen Startzeitpunkt den Fahrplan mit Kollisionen. Dann beginnt eine Schleife, in der chronologisch von der ersten Strecke bis zur letzten Strecke die erste vorkommende Kollision gesucht wird.

Im Folgenden wird die Iteration mit einer Startzeit von 35 betrachtet. Die Rückfahrt startet am Endbahnhof D zur Minute 35. Die Fahrtzeiten werden zunächst mitsamt den Kollisionen berechnet. Dann wird nach der chronologisch ersten Kollision gesucht.

Der Streckenabschnitt zwischen A und B wird auf Kollisionen geprüft. Das Intervall der Hinfahrt erstreckt sich hier von der Abfahrt in A zur Minute 0 bis zur Ankunft in B in der Minute 20. Die Rückfahrt plant, denselben Streckenabschnitt in der Gegenrichtung von der Abfahrt in B um 12 bis zur Ankunft in A zur Minute 32 zu belegen, was das Intervall [12; 32] ergibt. Da die Abfahrt der Hinfahrt (0) kleiner als die Ankunft der Rückfahrt (32) ist und gleichzeitig die Abfahrt der Rückfahrt (12) deutlich vor der Ankunft der Hinfahrt (20) stattfinden soll, liegt eine Kollision vor.

Um die Kollision zu korrigieren, wird anhand der Kostenfunktion nach der günstigsten Korrektur gesucht, d.h. es wird entweder auf der Hinfahrt oder auf der Rückfahrt Wartezeit an dieser Stelle eingebaut. Option A wäre, die Hinfahrt in Bahnhof A warten zu lassen: Die Rückfahrt kommt zur Minute 32 in A an, plus Einstiegszeit ergäbe das eine Wartezeit von 33 Minuten für die Hinfahrt. Option B wäre, die Rückfahrt in Bahnhof B warten zu lassen: Die Hinfahrt kommt zur Minute 20 in B an, plus Einstiegszeit bedeutet das eine Wartezeit von 9 Minuten für die Rückfahrt.

Die Kostenfunktion bewertet diejenige Wahl als günstiger, bei der die Wartezeit, die bei der Korrektur einzufügen ist, geringer ist, wobei ein Ungleichgewicht zwischen der Gesamtwartezeiten der Hin- und Rückfahrt mit eingerechnet wird. Da die Berechnung ergibt, dass Option B günstiger ist wählt der Algorithmus („greedy“) Option B als deutlich günstiger aus und baut 9 Minuten Wartezeit für die Rückfahrt am Bahnhof B ein.

Mit den geänderten Rückfahrtzeiten wird der Fahrplan neu berechnet. Die Rückfahrt wartet in B und verlässt den Bahnhof erst zur Minute 21. Die Strecke A-B hat nun für die Hinfahrt das Intervall [0; 20] und für die Rückfahrt das Intervall [21; 41]. Eine erneute Kollisionsprüfung ergibt, dass der Fahrplan an dieser Stelle nun stabil ist.

Die nächste gefundene Kollision tritt nun chronologisch auf dem Streckenabschnitt zwischen C und D auf. Das Intervall der Hinfahrt erstreckt sich hier von [42; 57]. Die Rückfahrt belegt denselben Abschnitt von [35; 50]. Da sich die Züge zeitlich überschneiden, wird erneut die Kostenfunktion angewendet. Option A wäre, die Hinfahrt in Bahnhof C warten zu lassen: Die Rückfahrt kommt zur Minute 50 an, plus Einstiegszeit ergäbe das eine Wartezeit von 9 Minuten für die Hinfahrt. Option B wäre, die Rückfahrt in Bahnhof D warten zu lassen, was eine Wartezeit von 23 Minuten bedeuten würde.

Da die Rückfahrt global bereits 9 Minuten wartet, ergibt sich aus der Kostenfunktion, dass Option A günstiger ist. Der Algorithmus wählt gierig Option A und fügt 9 Minuten Wartezeit auf der Hinfahrt in Bahnhof C ein.

Mit den geänderten Zeiten wird der Fahrplan neu berechnet. Die Hinfahrt verlässt Bahnhof C nun erst zur Minute 51, wodurch die Intervalle auf der Strecke C-D (Hinfahrt [51; 06], Rückfahrt [35; 50]) kollisionsfrei bleiben. Eine erneute Kollisionsprüfung ergibt, dass der gesamte Fahrplan nun an allen Stellen stabil ist.

Dieser Prozess wird wiederholt, bis ein stabiler Fahrplan ermittelt wurde oder die Maximalzahl an Versuchen für die aktuelle Verschiebung der Rückfahrt erreicht wurde. Ein Fahrplan gilt dann als stabil, wenn keine Kollisionen mehr vorkommen. Sollte ein stabiler Fahrplan gefunden werden, wird dieser mit dem aktuellen Bestergebnis verglichen. Die in dieser Iteration gefundene Lösung erreicht einen finalen Score von 162 (9^2 + 9^2 = 162) und schlägt das Fallback-Ergebnis des einseitigen Wartens (2116) deutlich, welches dann überschrieben wird. Am Ende wird der errechnete Fahrplan mit dem niedrigsten Score zurückgegeben.

1.2.7 Nicht-funktionale Anforderungen

Das System soll objekt-orientiert entworfen sein, d.h. konkret, dass das Programm in funktional getrennte Module aufgeteilt sein soll und austausch- und erweiterbare Teile haben. Ein zentraler Baustein ist die Nutzung des Strategy-Patterns. Dieses ermöglicht es, verschiedene Algorithmen zur Fahrplanberechnung über ein gemeinsames Interface als beliebig austauschbare Module einzubinden. Dadurch bleibt die Software hochgradig flexibel: Weitere Optimierungsstrategien können jederzeit nahtlos hinzugefügt werden, ohne die bestehende Kernlogik verändern zu müssen, diese als Klassen modelliert werden, die die FahrplanStrategie-Schnittstelle implementieren.

Außerdem soll das Programm laufzeittechnisch schnell laufen, da MATSE keine Programme nutzen kann, die zu lange brauchen, um Fahrpläne zu ermitteln.

2. Entwicklerdokumentation

2.1 Projektstruktur

Die Architektur der Anwendung folgt einem objektorientierten Design und ist in klar voneinander abgegrenzte Pakete (Packages) unterteilt. Dieser Aufbau nach dem Prinzip der „Separation of Concerns“ stellt sicher, dass Datenhaltung, Ein- und Ausgabelogik sowie die algorithmische Verarbeitung voneinander getrennt sind. Dies erhöht die Wartbarkeit des Codes und ermöglicht eine einfache Erweiterung des Systems. Das Projekt gliedert sich in drei zentrale Pakete „**model**“, „**io**“ und „**strategie**“ die durch eine übergeordnete Steuerungsschicht genutzt werden.

Das Paket „**eisenbahngesellschaft.model**“ bildet die zentrale Datenstruktur ab. Es enthält die informationstragenden Fachklassen „Bahnhof“ und „Strecke“. Diese Klassen bilden Bahnhöfe und die Verbindungsstrecken zwischen je zwei Bahnhöfen ab. Die Daten sind dabei so modelliert, dass eine Eisenbahnlinie durch eine Liste von „Strecke“-Objekten in der entsprechenden Reihenfolge dargestellt wird. Durch diese Datenstruktur ergibt sich, dass, außer dem Ersten und dem Letzten, alle Bahnhöfe doppelt abgespeichert werden. Bei der Verarbeitung muss also besondere acht darauf gelegt werden, dass die Daten konsistent bleiben.

Für die Kommunikation mit dem Dateisystem ist das Paket **eisenbahngesellschaft.io** zuständig. Es umfasst Klassen und Schnittstellen, um Eingabedateien zu parsen, die initialen Modellobjekte zu instanziieren und die final berechneten Fahrpläne in strukturierte Text-Dateien zu überführen. Durch diese Kapselung bleibt die restliche Anwendung unabhängig vom exakten Dateiformat.

Die Logik der algorithmischen Verarbeitung befindet sich im Paket „**eisenbahngesellschaft.strategie“**. Hier sind die verschiedenen Lösungsstrategien zur Fahrplanermittlung implementiert. Durch die Anwendung des „Strategy-Patterns“ implementieren die Algorithmen „Einfache Fahrt“, „Einseitiges Warten“ und „Beidseitiges Warten“ ein gemeinsames Interface. Die Strategien operieren dabei stets auf tiefen Kopien (Deep Copies) der Daten, um Seiteneffekte auf den Ursprungsfahrplan zu verhindern.

Der zentrale Einstiegspunkt der Anwendung ist die „**Main“**-Klasse. Ihre einzige Aufgabe besteht darin, die Systemargumente wie z.B. den Dateipfad zur Eingabedatei beim Programmstart entgegenzunehmen und den globalen Anwendungsprozess anzustoßen.

Die eigentliche Ablaufsteuerung findet in der Klasse „**FahrplanErmittler“** statt. Sie verbindet die drei isolierten Kernpakete miteinander: Zunächst wird eine Implementierung des „**InputHandlerInterface**“ beauftragt eine Textdatei einzulesen. Die wiedergegebenen Objekte übergibt er anschließend nacheinander an die verschiedenen Lösungsstrategien aus dem strategie-Paket. Sobald alle Strategien ihre Berechnungen abgeschlossen haben, sammelt der Controller die Ergebnisse und gibt sie an eine Implementierung des „**OutputHandlerInterface**“.

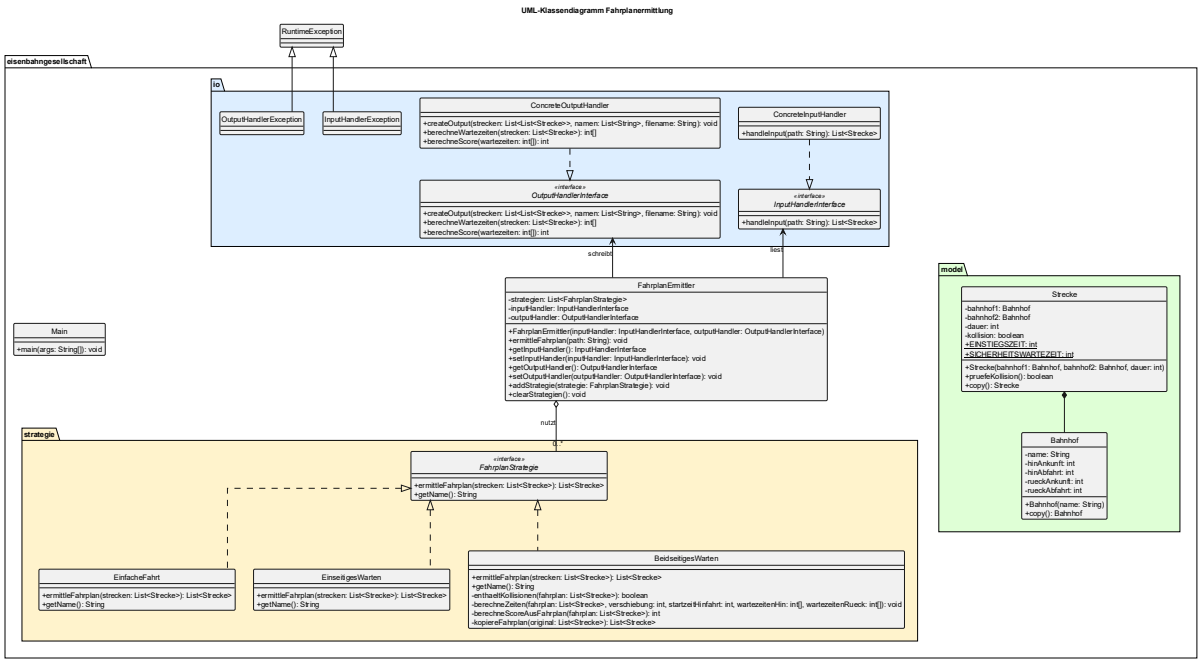
Spezifische Ausnahmesituationen, wie etwa fehlerhaft formatierte Eingabedateien, fehlende Verzeichnisse oder Schreibrechte bei der Ausgabe, werden über spezifische Ausnahmeklassen kontrolliert abgefangen und gesammelt in die Konsole ausgegeben.

2.2 Javadocs

Der Source-Code enthält Inline-Kommentare zur Erklärung der Arbeitsschritte. Zusätzlich gibt es JavaDocs, die unter „Dokumentation/javadocs/“ eingesehen werden können. Zum Öffnen sollte die index.html als Einstiegspunkt verwendet werden.

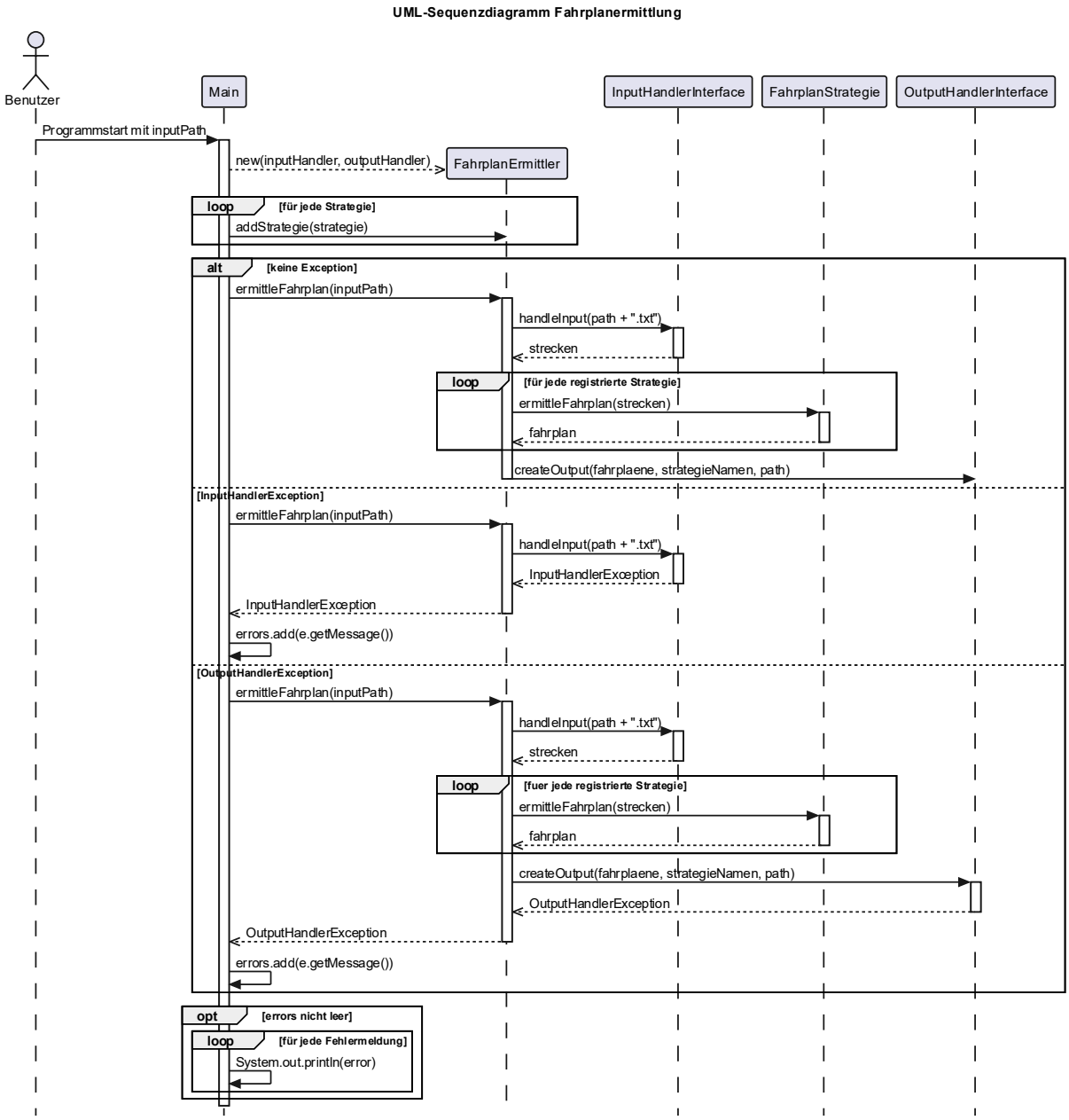
2.3 UML-Klassendiagramm

Ein UML-Klassendiagramm zur Projektstruktur lässt sich unter „Dokumentation\Diagramme\Klassendiagramm“ finden.



2.4 UML-Sequenzdiagramm

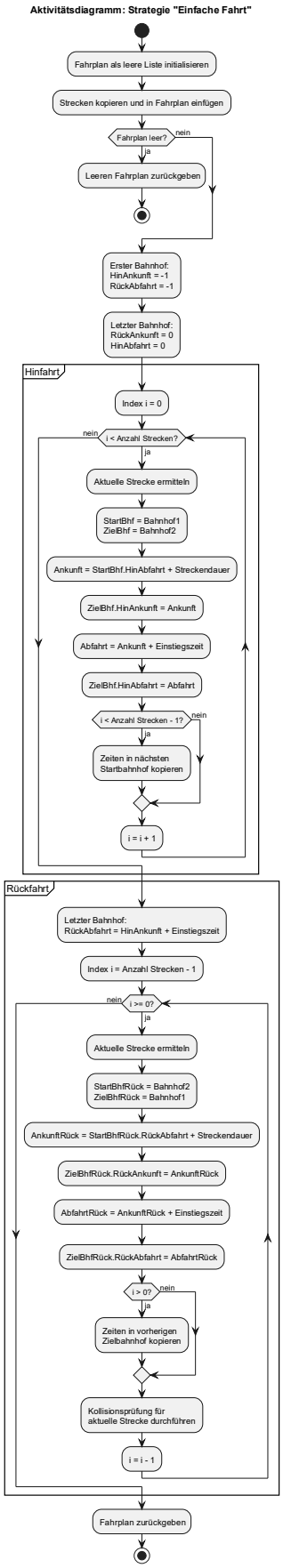
Ein UML-Sequenzdiagramm lässt sich unter „Dokumentation\Diagramme\Sequenzdiagramm“ finden.



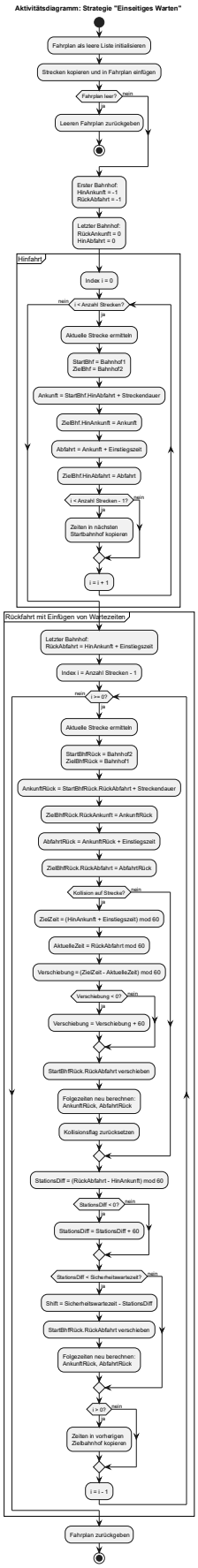
2.5 Aktivitätsdiagramme

Zur visuellen Veranschaulichung der Lösungsstrategien liegen unter „Dokumentation\Diagramme\Aktivitätsdiagramme“ Aktivitätsdiagramme vor.

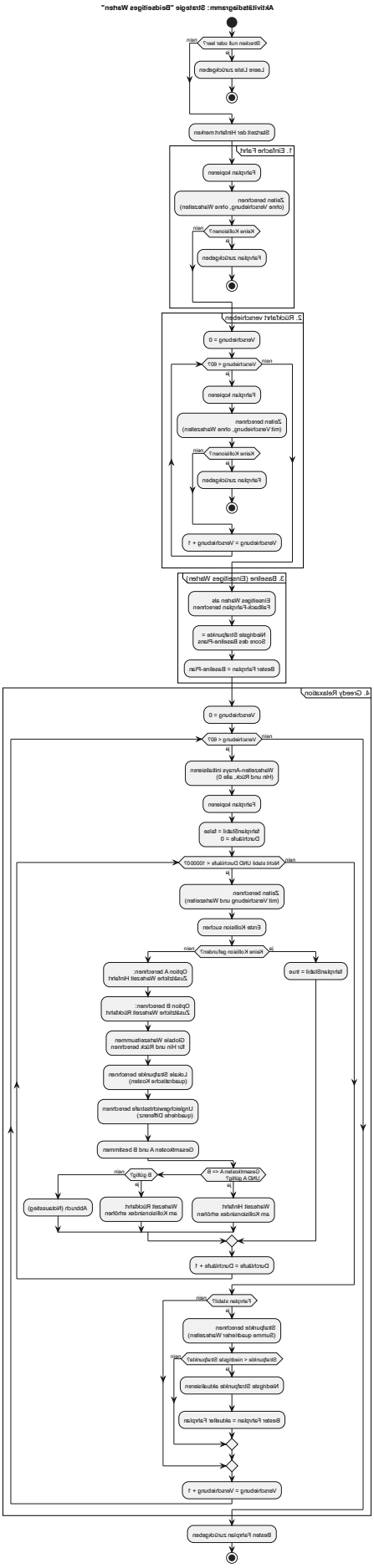
2.5.1 Einseitiges Warten



2.5.2 Beidseitiges Warten



2.5.3 Beidseitiges Warten


Aktivitätsdiagramm: Strategie "Beidseitiges Wachen"
Strecken mit oder leer?
nein
ja
Leere Liste zurückgeben
Startzeit der Hinfahrt merken
1. Einfache Fahrt
Fahrplan kopieren
Zeiten berechnen
(ohne Verschiebung, ohne Wartezeiten)
Keine Kollisionen?
nein
ja
Fahrplan zurückgeben
2. Rückfahrt verschieben
Verschiebung = 0
Verschiebung < 60?
nein
ja
Fahrplan kopieren
Zeiten berechnen
(mit Verschiebung, ohne Wartezeiten)
Keine Kollisionen?
nein
ja
Fahrplan zurückgeben
Verschiebung = Verschiebung + 1
3. Baseline (Einseitiges Wachen)
Einseitiges Wachen als
Fallback-Fahrplan berechnen
Niedrigste Strafpunkte =
Score des Baseline-Plans
Bester Fahrplan = Baseline-Plan
4. Greedy Relaxation
Verschiebung = 0
Verschiebung < 60?
nein
ja
Wartezeiten-Arrays initialisieren
(Hin und Rück, alle 0)
Fahrplan kopieren
fahrplanStabil = false
Durchlaufe = 0
Nicht stabil UND Durchlaufe < 100000?
nein
ja
Zeiten berechnen
(mit Verschiebung und Wartezeiten)
Erste Kollision suchen
Keine Kollision gefunden?
nein
ja
fahrplanStabil = true
Option A berechnen:
Zusätzliche Wartezeit Hinfahrt
Option B berechnen:
Zusätzliche Wartezeit Rückfahrt
Globale Wartezeitsummen
für Hin und Rück berechnen
Lokale Strafpunkte berechnen
(quadratische Kosten)
Ungleichgewichtszustrafe berechnen
(quadratische Differenz)
Gesamtkosten A und B bestimmen
Gesamtkosten A <= B
UND A gültig?
nein
ja
B gültig?
nein
ja
Wartezeit Hinfahrt
am Kollisionsindex erhöhen
Wartezeit Rückfahrt
am Kollisionsindex erhöhen
Abbruch (Notausstieg)
Durchlaufe = Durchlaufe + 1
Fahrplan stabil?
nein
ja
Strafpunkte berechnen
(Summe quadrierter Wartezeiten)
Strafpunkte < niedrigste Strafpunkte?
nein
ja
Niedrigste Strafpunkte aktualisieren
Bester Fahrplan = aktueller Fahrplan
Verschiebung = Verschiebung + 1
Besten Fahrplan zurückgeben

3. Testdokumentation

3.1 Diskussion der IHK-Beispiele

3.1.1 IHK-Beispiel 1

Das IHK-Beispiel 1 findet sich unter „input/Beispiel1.txt“ und seine entsprechende Ausgabedatei findet sich unter „output/output_Beispiel1.txt“. Das vom Programm ermittelte Ergebnis stimmt bei allen drei Strategien mit dem Muster-Ergebnis überein.

3.1.2 IHK-Beispiel 2

Das IHK-Beispiel 2 findet sich unter „input/Beispiel2.txt“ und seine entsprechende Ausgabedatei findet sich unter „output/output_Beispiel2.txt“. Das vom Programm ermittelte Ergebnis stimmt bei den Strategien „einfache Fahrt“ und „einseitiges Warten“ mit dem Muster-Ergebnis überein. Die Lösung, die vom Programm mit der Strategie „beiseitiges Warten“ bestimmt wurde, weist Unterschiede zum Muster-Ergebnis der IHK auf, allerdings erzielen beide Ergebnisse denselben Score und sind somit gleichwertig.

3.1.3 IHK-Beispiel 3

Das IHK-Beispiel 2 findet sich unter „input/Beispiel3.txt“ und seine entsprechende Ausgabedatei findet sich unter „output/output_Beispiel3.txt“. Das vom Programm ermittelte Ergebnis stimmt hier auch bei den Strategien „einfache Fahrt“ und „einseitiges Warten“ mit dem Muster-Ergebnis überein. Die Lösung, die vom Programm mit der Strategie „beiseitiges Warten“ bestimmt wurde, weist jedoch Unterschiede zum Muster-Ergebnis der IHK auf. Diesmal ist die vom Programm ermittelte Lösung zwar schlechter als das Ergebnis der IHK, ist allerdings trotzdem besser als die Lösung des einseitigen Wartens.

3.2 Eigene Beispiele

Zum Testen des Programms wurden eigene Beispiele geschrieben und vom Programm ausgeführt:

Eingabedatei	**Beschreibung**
Beispiel_Keine_Kollision.txt	Beispiel zur Demonstration des Verhaltens der drei Strategien, wenn keine Kollisionen vorkommen. Zu erwarten ist, dass alle drei Strategien dasselbe Ergebnis liefern.
Beispiel_Kollision_Verschiebung.txt	Zeigt das Verhalten der Algorithmen, wenn eine Kollision auftritt, die vom dritten Algorithmus durch Verschiebung der Rückfahrt gelöst werden kann. Zu erwarten ist, dass der Algorithmus die Kollisionen markiert, der Zweite Wartezeiten auf der Rückfahrt einbaut und der Dritte die Rückfahrt verzögert, um die Wartezeiten zu vermeiden.
Beispiel_Kollision_Greedy.txt	Zeigt das Verhalten des Dritten Algorithmus, wenn eine Kollision nicht durch Verschiebung der Rückfahrt aufgelöst werden kann. Zu erwarten ist, dass der dritte Algorithmus die Wartezeiten auf Hin-

	und Rückfahrt aufteilt, um den Score zu minimieren
Beispiel_Greedy_Groß.txt	Das Beispiel testet die Leistung des Programms an einem etwas größeren Problem. Die Lösung der dritten Strategie erzielt hier einen Score, der etwa einem 40stel des Scores der zweiten Strategie entspricht.

3.3 Grenz- und Fehlerfälle

3.3.1 Fehlerfälle

Fälle, in denen es zu Fehlern in der Fahrplanermittlung kommen kann, werden bereits bei der Eingabeverarbeitung entdeckt, dabei gibt es folgende Fehlerfälle:

Eingabedatei	Beschreibung	Erwartete Ausgabe	Tatsächliche Ausgabe
Beispiel_Fehler_Abstand_Zu_Groß.txt	Falls die Summe aus Abstand und Sicherheitszeit einer Strecke > 30 ist, ist es mathematisch nicht möglich eine kollisionsfreien Fahrzeitplan zu erstellen	Es sind Fehler aufgetreten: [FEHLER]: Die Summe aus Fahrtdauer und Sicherheitswartezeit darf nicht mehr als 30 Minuten sein. Bitte die Strecke: A -> B (32 min) überprüfen.	Es sind Fehler aufgetreten: [FEHLER]: Die Summe aus Fahrtdauer und Sicherheitswartezeit darf nicht mehr als 30 Minuten sein. Bitte die Strecke: A -> B (32 min) überprüfen.
Beispiel_Fehler_Abstand_Zu_Klein.txt	Die Fahrtdauer einer Teilstrecke muss echt größer 0 sein, da sonst anschaulich zwei Bahnhöfe ineinander liegen würden	Es sind Fehler aufgetreten: [FEHLER]: Die Fahrtdauer einer Strecke muss mindestens 1 Minute betragen. Bitte die Strecke: A -> B (0 min) überprüfen.	Es sind Fehler aufgetreten: [FEHLER]: Die Fahrtdauer einer Strecke muss mindestens 1 Minute betragen. Bitte die Strecke: A -> B (0 min) überprüfen.
Beispiel_Fehler_Bahnhofsanzahl.txt	Die Anzahl der angegebenen Bahnhöfe muss mindestens 2 betragen.	Es sind Fehler aufgetreten: [FEHLER]: Es müssen mindestens zwei Bahnhöfe angegeben werden.	Es sind Fehler aufgetreten: [FEHLER]: Es müssen mindestens zwei Bahnhöfe angegeben werden.
Beispiel_Fehler_Abstandsanzahl.txt	Die Anzahl der Abstände muss genau 1 weniger als die Anzahl der Bahnhöfe sein.	Es sind Fehler aufgetreten: [FEHLER]: Die Anzahl der Abstände passt	Es sind Fehler aufgetreten: [FEHLER]: Die Anzahl der Abstände passt

		nicht zur Anzahl der Bahnhöfe.	nicht zur Anzahl der Bahnhöfe.
Beispiel_Fehler_Falsches_Format.txt	Der Fehlerfall tritt auf, wenn zu wenige oder zu viele Zeilen vorhanden sind.	Es sind Fehler aufgetreten: [FEHLER]: Eingabedatei ist entweder leer oder hat nicht das erwartete Format (6 relevante Zeilen).	Es sind Fehler aufgetreten: [FEHLER]: Eingabedatei ist entweder leer oder hat nicht das erwartete Format (6 relevante Zeilen).
Beispiel_Fehler_Abstand_Keine_Zahl.txt	Der Fall tritt auf, wenn entweder einer der Abstände oder der Startzeitpunkt nicht als integer gelesen werden können.	Es sind Fehler aufgetreten: [FEHLER]: Fehler beim Parsen der Zahlenwerte (Dauer oder Startzeit).	Es sind Fehler aufgetreten: [FEHLER]: Fehler beim Parsen der Zahlenwerte (Dauer oder Startzeit).

3.3.2 Grenzfälle

Grenzfälle sind Beispielfälle, die genau an er Grenze zwischen zwei Äquivalenzklassen liegen. Für die oben beschriebenen Fehlerfälle lassen sich folgende Grenzfälle bestimmen:

Eingabedatei	**Beschreibung**	**Erwartete Ausgabe**	**Tatsächliche Ausgabe**
Beispiel_Grenze_abstand_1.txt	Der Abstand beträgt die größtmögliche Zahl, die noch valide ist. Mit einer Sicherheitszeit von 1 Minute, ist dieser Wert 29	Alle drei Strategien führen zu einem den Anforderungen entsprechenden Ergebnis	Alle drei Strategien führen zu einem den Anforderungen entsprechenden Ergebnis
Beispiel_Grenze_abstand_2.txt	Der Abstand beträgt die kleinstmögliche Zahl, die noch valide ist. Der Wert ist also gleich 1	Alle drei Strategien führen zu einem den Anforderungen entsprechenden Ergebnis	Alle drei Strategien führen zu einem den Anforderungen entsprechenden Ergebnis
Beispiel_Grenze_Bahnhof.txt	Die Anzahl der Bahnhöfe ist entspricht dem kleinstmöglichen validen Wert, also 2	Alle drei Strategien führen zu einem den Anforderungen entsprechenden Ergebnis	Alle drei Strategien führen zu einem den Anforderungen entsprechenden Ergebnis

4. Abschließende Bemerkungen

4.1 Zusammenfassung

Im Rahmen dieses Projekts wurde erfolgreich ein Softwaresystem zur automatisierten Fahrplangenerierung für eingleisige Eisenbahnstrecken entwickelt. Die primäre Herausforderung bestand darin, verschiedenen Algorithmen zur Erstellung eines streng getakteten 60-Minuten-Fahrplans zu implementieren, bei dem Kollisionen zwischen Zügen, die dasselbe Gleis zur selben Zeit befahren wollen, vermieden und notwendige Wartezeiten auf beide Fahrtrichtungen verteilt werden.

Zur Lösung dieses Problems wurde eine modulare und objektorientierte Java-Anwendung entwickelt. Durch den Einsatz des Strategy-Patterns wurden drei verschiedene Lösungsansätze implementiert: Die Strategie „Einfache Fahrt“, die Strategie „Einseitiges Warten“ sowie die Strategie „Beidseitiges Warten“. Insbesondere die Strategie des beidseitigen Wartens erfüllt die zentrale Kundenanforderung, da sie durch einen mehrstufigen Ansatz kollisionsfreie Fahrpläne generiert und die dabei entstehenden Wartezeiten möglich gleich auf die Hin- und Rückfahrt aufteilt.

Die saubere logische Trennung von Datenhaltung, Ein-/Ausgabeverarbeitung und Geschäftslogik stellt sicher, dass das System robust arbeitet, leicht zu warten ist und verlässliche Textberichte mit allen geforderten Leistungskennzahlen exportiert.

4.2 Ausblick

Die Implementierung des Strategie-Entwurfsmusters für die Fahrplanermittlung sorgt dafür, dass das Programm problemlos um weitere Strategien ergänzt werden kann. Sollte sich zum Beispiel ergeben, dass die MATSE-Eisenbahngesellschaft künftig doch Programme mit einer langen Laufzeit nutzen kann, könnten Ansätze zur Ermittlung von Fahrplänen mit Backtracking-Strategien diskutiert werden.

Falls die Laufzeit weiterhin ein entscheidender Faktor sein sollte, können stattdessen z.B. nach einer Hybrid-Lösung gesucht werden. Dabei könnte sich eine Strategie ergeben, bei der dynamisch die vorliegende Bahnlinie jeweils auf ihre Komplexität, also die Anzahl der Bahnhöfe und die Anzahl der Kollisionen, untersucht werden würde, um heuristisch abzuwägen, ob es sich um ein Problem handelt, welches laufzeitbedingt durch Backtracking lösbar ist. Falls die Komplexität den Kompetenzbereich des jeweiligen Backtracking-Algorithmus überschreitet, könnte die Strategie dann zu einem Greedy-Algorithmus umschalten. Es würde sich also eine Strategie ergeben, bei für kleine Probleme die Ideal-Lösung gefunden wird und falls das Problem in Größe wächst, wird ein gute Lösung in kurzer Laufzeit ermittelt.

4.3 Abweichungen und Ergänzungen zum Vorentwurf

Während der Implementierungsphase ergaben sich aus logischen Notwendigkeiten einige Anpassungen gegenüber dem initialen, handschriftlichen Konzept.

4.3.1 Ergänzungen

Die „Main“-Klasse wurde im Konzept nicht näher thematisiert. Sie stellt den zentralen Einstiegspunkt in das Programm dar. Sie bekommt den Pfad zur Eingabedatei zusammen mit einem Paar optionaler Parameter übergeben und stößt die Fahrplanermittlung im „FahrplanErmittler“ an. Sie kümmert sich außerdem um die Fehlerbehandlung, indem Fehlermeldungen gesammelt und in die Konsole ausgegeben werden.

Gemäß der Anforderung, dass Einstiegs- und Sicherheitswartezeiten ohne Mühe anpassbar sein sollen, wurde die „Strecke“-Klasse um entsprechende statische Attribute, mit passenden Setter-Methoden erweitert.

Im ConcreteInputHandler wurden spezifischere Validierungen der Eingabe Parameter ergänzt, die im Vorentwurf nicht explizit modelliert waren. So wird nun geprüft, ob die Startzeit der Hinfahrt im Intervall [0; 59] liegt, ob die Abstände und die Hinfahrtszeit als ganzzahlige Werte übergeben wurden, ob das Format der Eingabedatei eingehalten wurde, ob die Anzahl der Bahnhöfe und der Abstände logisch sinnvoll ist, ob der Wert für die Startzeit logisch sinnvoll ist und ob die Summe aus Fahrtdauer und Sicherheitswartezeit 30 Minuten überschreitet. Letzteres ist mathematisch zwingend notwendig, da eine Belegung von über 30 Minuten in einem 60-Minuten-Takt unweigerlich zu unlösbaren Deadlocks auf eingleisigen Strecken führt.

Die Klasse „FahrplanErmittler“ wurde um die Möglichkeit zum Hinzufügen neuer Strategien ergänzt, damit das Strategy-Pattern sinnvoll genutzt werden kann.

4.3.2 Abweichungen

Das ursprüngliche Konzept skizzierte für das „Beidseitige Warten“ eine reine iterative Verteilung der Wartezeiten. Das Problem mit dieser Lösung waren die zirkulären Bezüge der Kollisionen und der fehlende Ansatz zur Verschiebung der Rückfahrt. In der tatsächlichen Implementierung wurde der Algorithmus stark angepasst, um ein tatsächliches Ergebnis zu liefern. Er arbeitet nun in drei Eskalationsstufen: Zunächst probiert das Programm die einfache Fahrt, sollte es dabei zu Kollisionen kommen, prüft das System, ob eine simple Verschiebung der Startzeit der Rückfahrt das Problem löst. Für den Fall, dass auch dieser Ansatz scheitert, wurde ein Greedy-Relaxation-Algorithmus implementiert, der versucht das Problem zu lösen. Da dieser Ansatz nicht zwingend einen kollisionsfreien Fahrplan findet, wird das Ergebnis des einseitigen Wartens als Notlösung verwendet.

Die Logik zur Ermittlung der Fahrpläne für jede Strategie wurde im FahrplanErmittler von den geplanten zwei Methoden in eine einzige Methode zusammengefasst, da sich ergeben hat, dass keine Hilfsmethode notwendig ist.

4.4 Benutzeranleitung

Die Abgabe ist wie folgt aufgebaut:

```
│ run.sh
│ eisenbahngesellschaft.jar
│
├───Dokumentation
│       <Dateien der Dokumentation>
├───input
│       <Beispiel-Eingabedateien>
├───jar
│       eisenbahngesellschaft.jar
├───output
│       <Ausgabedateien, die durch das Programm generiert wurden>
└───src
        └───main
                └───java
                        └───eisenbahngesellschaft
                                ├───io
                                │       ConcreteOutputHandler.java
                                │       ConcreteInputHandler.java
                                │       InputHandlerException.java
                                │       OutputHandlerException.java
                                │       InputHandlerInterface.java
                                │       OutputHandlerInterface.java
                                ├───model
                                │       Bahnhof.java
                                │       Strecke.java
                                │
                                ├───strategie
                                │       BeidseitigesWarten.java
                                │       EinfacheFahrt.java
                                │       EinseitigesWarten.java
                                │       FahrplanStrategie.java
                                │
                                │ FahrplanErmittler.java
                                │ Main.java
```

Das kompilierte Programm ist unter jar/eisenbahngesellschaft.jar zu finden. Das Shell-Skript „run.sh“ führt das Programm für alle Eingabedateien im Ordner „input/“ aus. Zur Ausführung dieses Skripts wird eine Java21-JDK und eine GIt-Bash vorausgesetzt. Bei Bedarf können weiter Beispiele in den Eingabeordner gelegt werden, um diese auszuführen, wobei Dateien, die nicht auf „.txt“ enden von dem Programm nicht als valide Eingabe erkannt werden.

Außerdem kann die .jar Datei selbst über einen Konsolen Befehl ausgeführt werden, falls die Sicherheitswartezeit oder die Einstiegszeit angepasst werden sollen. Dafür kann das

Programm mit 'java -jar jar/eisenbahngesellschaft.jar "<Pfad zur Eingabedatei ohne Dateiendung>" <Sicherheitswartezeit> <Einstiegszeit>' ausgeführt werden.

Die Ergebnisse eines Programmdurchlaufs werden im Ordner „output/“ als .txt Datei abgelegt.

Die „eisenbahngesellschaft.jar“ liegt sowohl im jar Ordner als auch als Kopie auf oberster Ebene vor.

4.5 Verwendete Hilfsmittel

- Git-Bash
- Maven
- Jetbains IntelliJ IDE
- PlantUML
- Microsoft Word

4.6 Entwicklungsumgebung

Programmiersprache: Java

Compiler: Open Oracle JDK-21

Betriebssystem: Windows 11

4.7 Eigenhändigkeitserklärung

Ich erkläre verbindlich, dass das vorliegende Prüfungsprodukt von mir selbstständig erstellt wurde. Die als Arbeitshilfe genutzten Unterlagen sind in der Arbeit vollständig aufgeführt. Ich versichere, dass der vorgelegte Ausdruck mit dem Inhalt der von mir erstellten digitalen Version identisch ist. Weder ganz noch in Teilen wurde die Arbeit bereits als Prüfungsleistung vorgelegt. Mir ist bewusst, dass jedes Zuwiderhandeln als Täuschungsversuch zu gelten hat, der die Anerkennung des Prüfprodukts als Prüfungsleistung ausschließt.

Name: Mohammed Alassane

Köln, den 08.05.2026

Unterschrift